

GSoC 2026 · CERN-HSF / ROOT

BatchNorm2D Parser for SOFIE

TMVA::Experimental::SOFIE::PyTorch · Exercise 4 – Operator Expansion

Author: **Aditya Rathore** · GitHub: @AdityaDRathore

Submitted: March 2026 · For review by Lorenzo Moneta & Sanjiban Sengupta

§0 Context & Motivation

This document is my technical write-up for **Exercise 4** of the GSoC 2026 CERN-HSF SOFIE evaluation. The exercise asks candidates to extend the SOFIE PyTorch parser with parsing functionality for new operator types. I chose to begin with **BatchNorm2D** because it sits at an interesting intersection of graph-topology complexity, memory-layout safety, and physics-inference correctness — all of which are first-class concerns inside SOFIE's strictly-typed, statically-allocated C++ backend.

My prior work on the project (PR #21528, adding 10 new operators to RModelParser_PyTorch.cxx) gave me direct exposure to the pybind11/C-API boundary and the strict serialization contracts that SOFIE's factory methods impose. That groundwork informed every decision described here.

§1 What is BatchNorm2D — and Why Is It Non-Trivial to Parse?

1.1 The Operator

PyTorch's `nn.BatchNorm2d` normalises an NCHW tensor channel-wise during training, then at inference uses frozen running statistics. The affine transformation applied at inference time is:

$$y_c = (x_c - \mu_c) / \sqrt{(\sigma_c^2 + \epsilon)} \cdot \gamma_c + \beta_c$$

where γ (weight / scale) and β (bias) are the learned affine parameters, μ and σ^2 are the running statistics frozen after training, and ϵ is a small numerical-stability constant. The spatial dimensions H , W are averaged away — all four state vectors are 1-D arrays of length C , the channel count.

1.2 Why Parsing Is Non-Trivial

From the parser's point of view, three flags on `nn.BatchNorm2d` fundamentally change what appears in the traced ONNX graph:

Flag	Graph Effect	Parser Action Required
affine=True (default)	γ and β appear as ONNX initializer edges	Read from initializers dict
affine=False	No scale/bias edges in graph	Synthesise $\gamma=1$, $\beta=0$ arrays of shape [C]
track_running_stats=False	μ and σ^2 edges absent; dynamic batch stats at runtime	Hard reject — mathematically undefined for batch_size=1

§2 Design Decisions — The 'Why'

2.1 Do Not Pre-Inject ϵ into the Variance Tensor

My first instinct when extracting σ^2 was to fuse epsilon in Python and hand the C++ side a pre-computed $\sigma^2 + \epsilon$ array. That would be wrong. Looking at `ROperator_BatchNormalization::Initialize()` in the ROOT source, the C++ macro computes:

ROperator_BatchNormalization.hxx — fused scale computation

```
fused_scale_data[i] = original_scale[i] / std::sqrt(original_var[i] + fepsilon);
```

If I had pre-injected ϵ in Python, the C++ code would add it again, computing $\sigma^2 + 2\epsilon$ under the square root. At HEP inference precision levels, that is a measurable bias. The parser therefore ships the raw σ^2 unchanged and lets the C++ side own the numerical-stability fusion — which is exactly the responsibility boundary SOFIE was designed around.

2.2 training_mode Must Be Hardcoded to Zero

SOFIE is a pure inference engine. The ONNX BatchNormalization opset-9 node carries a `training_mode` attribute, but SOFIE only supports the evaluation path. Rather than reading the attribute and hoping it's correct, I hardcode `training_mode: 0` in the serialisation dictionary. This is an intentional, explicit contract: the parser tells the C++ factory 'lock this node into static evaluation mode', preventing any runtime branch that might attempt to update running statistics.

Design principle: When a field's value is contractually fixed for all valid use-cases, hard-coding it is safer than reading it — it converts a silent bug into an impossible state.

2.3 The track_running_stats=False Hard Reject

This was the most important design decision in the entire module. When `track_running_stats=False`, PyTorch calculates μ and σ^2 dynamically from the current mini-batch at runtime. In a training loop with `batch_size ≥ 32` this is statistically well-defined. SOFIE, however, frequently runs single-event inference (`batch_size=1` is common in ROOT analysis workflows). At `batch_size=1`:

- The per-batch mean equals the single sample value: $\mu = x$.
- The per-batch variance is zero: $\sigma^2 = 0$.
- The normalised output becomes: $y = (x - x) / \sqrt{0 + \epsilon} \cdot \gamma + \beta = \beta$.

The input signal is completely erased. Any downstream physics discriminant would be computing on a tensor of constants. My first attempt was to synthesise `mean=0, var=1` as fallback arrays — this would have prevented the C++ crash but would have silently produced *wrong* physics results with no error. That is the worst possible failure mode for an inference engine.

Final decision: throw a descriptive `ValueError`. An inference engine must fail loudly rather than fail silently. Silent data corruption in physics analysis is infinitely worse than a hard abort.

2.4 Memory Contiguity Enforcement

PyTorch tensors manipulated in Python can silently become non-contiguous strided views — for example, after transpose operations or slice indexing. Passing such a tensor's data pointer through the pybind11 bridge to a C++ `float*` results in a segmentation fault because the C++ code assumes a flat memory stride of `sizeof(float)` per element, which strided views violate.

Memory contiguity enforcement — applied to every tensor crossing the Python/C++ boundary

```
return np.ascontiguousarray(tensor, dtype=np.float32)
```

This single call guarantees three things: the array is C-contiguous, its element type is exactly `float32` (matching `float` on all target architectures), and if the tensor was already contiguous the operation is a no-op (zero cost).

§3 Implementation Walkthrough

3.1 Function Signature and Graph Contracts

The parser function mirrors the calling convention established by the existing PyTorch parser infrastructure:

parse_batchnorm2d_node — function signature

```
def parse_batchnorm2d_node(node, initializers, node_dict):
    """
    node          : PyTorch ONNX IR Node (onnx::BatchNormalization)
    initializers: {debugName -> np.ndarray} (weights loaded from state_dict)
    node_dict     : {debugName -> Node}      (for onnx::Constant lookup)
    """
```

The three-argument contract reflects how SOFIE's C++ parser constructs the Python environment via `PyRunString`: the initializer dictionary is populated from the model's `state_dict()` and the `node_dict` maps every output `debugName` to its producing node, enabling constant-folded attribute lookup.

3.2 Attribute Extraction

ONNX node attributes are accessed via a kind-dispatch helper that avoids hardcoding the PyTorch internal attribute type string. The fallback values match PyTorch defaults:

Attribute dispatch — epsilon and momentum

```
def _node_get(node, key):
    sel = node.kindOf(key)      # returns 'f', 'i', 's', etc.
    return getattr(node, sel)(key)

epsilon = _node_get(node, 'epsilon') if node.hasAttribute('epsilon') else 1e-05
momentum = _node_get(node, 'momentum') if node.hasAttribute('momentum') else 0.9
```

3.3 Tensor Retrieval: Initializers and Constants

Each input edge in the ONNX graph is identified by its debugName. Tensors can live in two places: the initializers dictionary (for trained weights) or as `onnx::Constant` nodes in the graph (for compile-time constants). The retrieval helper handles both, with CUDA safety on the constant path:

Tensor retrieval — initializer and onnx::Constant paths

```
def get_tensor_data(input_edge):
    name = input_edge.debugName()
    if name in initializers:
        return np.ascontiguousarray(initializers[name], dtype=np.float32)
    elif name in node_dict:
        src = node_dict[name]
        if src.kind() == 'onnx::Constant':
            t = _node_get(src, 'value')
            return np.ascontiguousarray(
                t.detach().cpu().numpy(), dtype=np.float32
            )
    return None
```

3.4 Channel Inference

Knowing the channel count C is necessary to synthesise fallback arrays for `affine=False` models. I infer it in two stages: first from the type information on the input X edge (`sizes()[1]`), then, if that returns a `SymInt` (dynamic shape), from the shape of whichever tensor was successfully retrieved:

Channel inference with SymInt guard

```
x_type = inputs[0].type()
channels = None
if x_type is not None and x_type.kind() == 'TensorType':
    sizes = x_type.sizes()
    if sizes and len(sizes) >= 2:
        channels = int(sizes[1])  # cast SymInt -> int

# Fallback: infer from whichever tensor we did retrieve
if channels is None:
    for t in [scale, bias, mean, var]:
        if t is not None:
            channels = int(t.shape[0])
            break
```

3.5 Serialisation Dictionary

The final return value is a strictly-typed dictionary matching the key-set expected by the SOFIE C++ factory's `PyDict_GetItemString` calls:

Serialisation dictionary — SOFIE C++ factory contract

```

return {
    'type'      : 'BatchNorm2d',
    'name'      : node_name_clean,    # dots replaced with underscores
    'num_features' : int(channels),
    'eps'       : float(epsilon),
    'momentum'   : float(momentum),
    'training_mode' : 0,              # hardcoded – SOFIE is inference-only
    'running_mean' : mean,            # np.ndarray, C-contiguous, float32
    'running_var'  : var,
    'weight'      : scale,
    'bias'        : bias,
}

```

§4 Bugs Encountered During Development

1. Null Pointer Dereference in ROperator Initialisation

Root cause: C++ segfault inside BatchNormalization initialisation. The initial payload dictionary omitted the momentum, training_mode, type, and name keys. When pybind11 executed PyDict_GetItemString on those missing keys, it returned null pointers which the C++ backend dereferenced without validation.

Fix: Integrated all required keys. Hard-coded training_mode: 0. Cleaned the name string (dots → underscores) to avoid C++ namespace collisions.

2. torch.SymInt Cannot Be Interpreted as int

Root cause: TypeError crash inside np.ones(channels) when tracing a dynamically-batched model. PyTorch's newer export path represents tensor dimensions as symbolic integers (SymInt) rather than concrete Python ints. NumPy does not accept SymInt as an array size argument.

Fix: Enforced explicit casting at every dimension extraction point: channels = int(sizes[1]).

3. CUDA / Autograd Contamination

Root cause: RuntimeError: Can't call numpy() on Tensor that requires grad. Tensors inside onnx::Constant nodes may still be attached to the autograd graph or reside on a CUDA device, triggering an exception when .numpy() is called directly.

Fix: Applied the three-step sanitisation chain universally: tensor.detach().cpu().numpy(). detach() severs the autograd graph, .cpu() moves the data to host memory, and only then can NumPy safely wrap the pointer.

4. Silent Physics Corruption via Synthesised Statistics

Root cause: Initially I synthesised $\text{mean}=0$, $\text{var}=1$ when $\text{track_running_stats}=\text{False}$ to prevent a C++ crash. This appeared to work — the model ran. However, it silently replaced the real data distribution with an identity transform ($y = x \cdot \gamma + \beta$), corrupting all downstream physics discriminants with no error message.

Fix: Removed the synthesis hack entirely and replaced it with a hard `ValueError`. The error message explicitly names the unsupported configuration so the user knows exactly what went wrong and why.

§5 Validation & Test Coverage

I wrote a companion unittest module, `test_parse_batchnorm2d.py`, that constructs real PyTorch modules, traces them through `_model_to_graph`, and validates every aspect of the parser output programmatically. All three tests pass (3/3, 0.066 s).

Test Case	What It Validates	Result
<code>test_standard_batchnorm</code>	Correct type string, <code>num_features=6</code> , <code>ε=1e-5</code> , <code>momentum=0.1</code> , all four arrays C-contiguous float32	PASS
<code>test_affine_false_batchnorm</code>	Synthesised weight == <code>ones(6)</code> , bias == <code>zeros(6)</code> when <code>affine=False</code>	PASS
<code>test_track_running_stats_false</code>	<code>ValueError</code> raised with message containing 'Dynamic BatchNormalization (<code>track_running_stats=False</code>) is unsupported'	PASS

All three tests pass cleanly. The only terminal output from the test run is the expected ONNX export `UserWarning` about `training_mode` (this is a PyTorch 2.x behaviour, not a test failure), plus the three-dot success line from unittest.

§6 Lessons Learned

These are the things I now understand better having worked through this exercise — not just the mechanics, but the engineering philosophy behind them.

- **Memory safety is not optional at language boundaries:** The Python↔C++ bridge through `pybind11` is a trust boundary. The Python side has garbage collection, reference counting, and strided memory views. The C++ side has raw pointers and static allocation. If you pass a strided pointer across that boundary you get a segfault. `np.ascontiguousarray` is the toll booth.
- **Never satisfy an API signature by faking mathematical state:** The `track_running_stats=False` synthesis bug taught me the most important lesson: making a function not crash is not the same as making it correct. The synthesised $\text{mean}=0$ / $\text{var}=1$ fallback was worse than a hard crash because it produced wrong answers that looked right. In physics inference, silent correctness failures corrupt downstream analysis silently. Hard aborts are the ethical choice.
- **Know where the arithmetic boundary lies between Python and C++:** The epsilon pre-fusion bug was subtle. The C++ `ROperator` owns the fused-scale computation for a reason — it controls the numerical

precision of that operation. The parser's job is to faithfully transport weights, not to optimise them. Respecting that boundary kept the mathematics clean.

- **SymInt is not an int, and int() is not free:** PyTorch 2.x's symbolic shape system is powerful but it breaks the assumption that dimension values are Python ints. The explicit `int()` cast is a one-liner that prevents a cryptic crash in production and signals to future readers that this boundary is intentional.

§7 Relation to Existing SOFIE Infrastructure

This Python parser is designed to plug into the existing SOFIE PyTorch pipeline at the point where `RModelParser_PyTorch.cxx` dispatches to Python via `PyRunString` after identifying an `onnx::BatchNormalization` node kind. The returned dictionary is consumed by the C++ factory in exactly the same way as the dictionaries returned by the existing `Conv2D` and `Linear` parsers.

The `training_mode`, `eps`, `momentum`, and `num_features` keys map directly to the constructor arguments of `ROperator_BatchNormalization`. The four numpy arrays (`weight`, `bias`, `running_mean`, `running_var`) are passed to the corresponding `Initialize()` vectors via the `pybind11` buffer protocol, which requires C-contiguous `float32` layout.

§8 Next Steps

`BatchNorm2D` is one of six operators listed in Exercise 4. My plan for the remaining operators follows the same pattern — trace the ONNX graph, map inputs to initializers, handle edge-cases — but each brings new complexity:

- **ELU:** Attribute-only node (`alpha`), no weight tensors. Straightforward.
- **MaxPool2D:** Requires `kernel_shape`, `strides`, `pads`, `dilations` attribute extraction.
- **RNN / GRU / LSTM:** Recurrent operators with multi-output edges, `bidirectionality` flags, and packed weight matrix layout — the most complex of the six. I plan to tackle these after `MaxPool2D`.

All code is staged on my personal development branch and is ready for upstream review. I have not opened a PR to the ROOT project per the exercise instructions.

Aditya Rathore

GSoC 2026 Candidate — SOFIE PyTorch/Keras Parser

GitHub: @AdityaDRathore · adityarathore7067@gmail.com

Submitted March 2026